

A proposition as a function

A proposition is a function f^{prop} . We define it as a sequence

$$f^{\text{prop}} = \langle f_1, f_2, \dots, f_n \rangle,$$

meaning it is evaluated step by step:

- The domain of f_1 lies in the domain of f^{prop} ; apply the relevant terms to get $\text{cod}(f_1)$.
- For each k , the domain of f_k lies in $\text{dom}(f^{\text{prop}}) \cup \text{cod}(f_1) \cup \dots \cup \text{cod}(f_{k-1})$; apply the relevant terms to get $\text{cod}(f_k)$.
- This continues until $\text{cod}(f_n) = \text{cod}(f^{\text{prop}})$, the goal.

Equivalently:

$$\text{dom}(f_k) \subseteq \text{dom}(f^{\text{prop}}) \cup \bigcup_{j < k} \text{cod}(f_j), \quad k = 1, \dots, n, \quad \text{cod}(f_n) = \text{cod}(f^{\text{prop}}).$$

The union above is imprecise: it says f_k 's inputs lie *somewhere* in everything proved so far, but a step actually uses a *specific* set of prior facts. The two forms below name those dependencies exactly.

The standard way: a numbered derivation

Fix the hypotheses $H = (h_1, \dots, h_m)$ (the “given” of the proposition). A proof is a numbered list of facts. Each line states a fact S_k , the rule f_k that produces it, and the exact earlier lines / hypotheses it consumes:

$$\begin{array}{lll} s_1 : S_1 & \text{by } f_1 & \text{from } \{\text{some of } H\} \\ s_2 : S_2 & \text{by } f_2 & \text{from } \{\text{some of } H, s_1\} \\ \vdots & \vdots & \vdots \\ s_k : S_k & \text{by } f_k & \text{from } D_k \subseteq \{H, s_1, \dots, s_{k-1}\} \\ \vdots & \vdots & \vdots \\ s_n : S_n & \text{by } f_n & \text{from } D_n, \quad S_n = G. \end{array}$$

Here D_k is the *precise* list of inputs to step k (this is what your `stepk` file records as its binders), the goal is G , and the proof is complete because the last fact S_n is G . This is the same sequence $\langle f_1, \dots, f_n \rangle$ as above — only the vague union $\bigcup_{j < k} \text{cod}(f_j)$ is replaced by the exact dependency set D_k .

The same thing as a graph

Equivalently, the proof is a directed acyclic graph. Nodes are the facts $H, s_1, \dots, s_n$; a node s_k has an incoming edge from each fact in its dependency set D_k :

$$d \longrightarrow s_k \quad \text{for every } d \in D_k, \quad s_n = G \text{ (the sink)}.$$

The numbered list above is just one topological ordering of this graph. Writing $f^{\text{prop}} = \langle f_1, \dots, f_n \rangle$ picks such an ordering and then forgets the individual edges; the graph (or the list with its D_k 's) keeps them.

The fully formal way: the proof term

This is the precise object Lean stores. Nothing is a black box: every step names the lemma it uses and the exact arguments it is applied to.

Notation, each symbol explained

$t : T$ “ t has type T .” Since a proposition *is* a type, this reads “ t is a proof of the proposition T .” E.g.
 $h : A = B$ means h is a proof that $A = B$.

$A \rightarrow B$ the type of *functions* from A to B . Read as a proposition it is the implication “ A implies B ”: a function that turns a proof of A into a proof of B .

$\prod_{x:A} B(x)$ a *dependent* function type (“for all $x : A$, $B(x)$ ”). Same as $A \rightarrow B$ except the output type $B(x)$ may mention the input x . This is how “for all points $x, \dots$ ” is written. When B does not mention x it collapses to $A \rightarrow B$.

$f a$ *application*: the function f applied to the argument a . Just juxtaposition, no parentheses needed; $f a b$ means $(f a) b$, i.e. f applied to a then to b . This is the “what is applied to what.”

$\lambda x. e$ *lambda abstraction*: the function that takes an input x and returns e (where e may use x). The dot “.” just separates the input variable x from the body e . So $\lambda x. x$ is the identity function, and $\lambda h. (\dots h \dots)$ is “given a proof h , build $\dots h \dots$ ”

let $x := e_1$ **in** e_2 name the value e_1 as x , then use x inside e_2 . Pure abbreviation: it is exactly e_2 with x standing for e_1 . This is what lets us build one intermediate fact at a time.

$\langle a, b \rangle$ a *pair* (tuple). As a proof, $\langle a, b \rangle$ proves a conjunction “ $A \wedge B$ ” by pairing a proof $a : A$ with a proof $b : B$. Used at the end to assemble several facts into the goal.

The proof term

Let the proposition have hypotheses $h_1 : H_1, \dots, h_m : H_m$ and goal G . Its type (the statement) is

$$f^{\text{prop}} : \prod_{h_1:H_1} \dots \prod_{h_m:H_m} G,$$

and its proof (the term after “ $:=$ ”) is

$$\begin{aligned} f^{\text{prop}} &:= \lambda h_1 \dots h_m. \text{let } s_1 := \ell_1(a_{1,1}, \dots, a_{1,r_1}) \text{ in} \\ &\quad \text{let } s_2 := \ell_2(a_{2,1}, \dots, a_{2,r_2}) \text{ in} \\ &\quad \vdots \\ &\quad \text{let } s_n := \ell_n(a_{n,1}, \dots, a_{n,r_n}) \text{ in} \\ &\quad s_n. \end{aligned}$$

Reading it: each ℓ_k is a *named* lemma (a cited proposition or an axiom), and each argument $a_{k,j}$ is a *specific* earlier fact — one of the hypotheses $h_1, \dots, h_m$ or an earlier step $s_1, \dots, s_{k-1}$. So $s_k := \ell_k(a_{k,1}, \dots, a_{k,r_k})$ says exactly “apply lemma ℓ_k to these particular inputs to obtain fact s_k .” The vague f_k from before is now the concrete pair (name ℓ_k , argument list $a_{k,\bullet}$), and the dependency set is $D_k = \{a_{k,1}, \dots, a_{k,r_k}\}$. The final line returns s_n , whose type is the goal G .

A concrete step

For instance a single step applying Euclid’s Proposition 4 (SAS) might read

$$\text{let } s_5 := \text{proposition_4 } (h_1) (s_2) (s_3) \text{ in } \dots$$

meaning: the lemma is **proposition_4**; it is applied to hypothesis h_1 and to the earlier facts s_2 and s_3 ; the result is named s_5 . And if the goal G is a conjunction $G_1 \wedge G_2$ proved by facts s_i and s_j , the last line is $\langle s_i, s_j \rangle$ instead of a single s_n .

Naming the sequence of steps: the derivation

The term f^{prop} is a *single* object $\lambda \vec{h}. (\mathbf{let} \dots \mathbf{in} s_n)$. Once assembled it is atomic: the **lets** reduce away, so there is *no* projection $\pi_k(f^{\text{prop}})$ that returns “the k -th step.” To speak of the ordered list of steps we keep a separate, inspectable object and give it a name — its *derivation*.

New notation

σ_k the k -th *step*, i.e. the triple $\sigma_k = \langle \ell_k, D_k, S_k \rangle$: the lemma ℓ_k used, its dependency set $D_k \subseteq \{\vec{h}, s_1, \dots, s_{k-1}\}$ (the exact inputs), and the fact S_k it produces ($s_k : S_k$).

$\mathcal{D}$ the *derivation*: the ordered tuple of all steps,

$$\mathcal{D} = (\sigma_1, \sigma_2, \dots, \sigma_n).$$

Unlike f^{prop} , this *is* indexable — $\sigma_k, \ell_k, D_k, S_k$ all make sense.

$\llbracket \cdot \rrbracket$ the *assembly* (elaboration) map, which collapses a derivation into a single proof term:

$$f^{\text{prop}} = \llbracket \mathcal{D} \rrbracket = \lambda \vec{h}. \mathbf{let} s_1 := \ell_1(D_1) \mathbf{in} \dots \mathbf{let} s_n := \ell_n(D_n) \mathbf{in} s_n.$$

The point

$\mathcal{D}$ and f^{prop} are *different objects*: $\mathcal{D}$ is an ordered, inspectable list of steps, while $f^{\text{prop}} = \llbracket \mathcal{D} \rrbracket$ is the flat term it elaborates to. You refer to individual steps through $\mathcal{D}$ (as σ_k, D_k, S_k), never by trying to index into f^{prop} . In our pipeline each backing file **step_k** *is* a σ_k (its binders are D_k , its theorem type is S_k), and wiring the proof is exactly computing $\llbracket \mathcal{D} \rrbracket$.